



Solutions Architect, Technical Screen

 Candidate handout. We're excited to keep moving forward here!

Overview

- **Prep:** you have **3 days** (take-home)
- **Session:** 45 minutes, live walkthrough + a few questions

This is a focused technical screen. We want to see you build something real and reason about your choices.

The scenario

A customer's platform team is onboarding new engineers onto a large, convention-heavy internal library, and ramp is taking too long. They've asked you to build something that gets a new engineer productive faster and keeps their code consistent with the team's standards. They care about the whole path a change takes, plan, design, build, review, test, and deploy, not just the first commit, and they'd like something the people around engineering can use too, PMs, QA, and DevOps. Part of the exercise is shaping one solution that serves multiple audiences, not just an individual coder.

Since you won't have their private codebase, **choose a well-documented open-source library as a stand-in** and build against it. Good options: HuggingFace `diffusers`, OpenCV (Python), MONAI, scikit-image, or any library you know well and can defend.

What the customer asked for (requirements)

Build something that helps a new engineer:

1. **Scaffold a correct first contribution** following the library's conventions, without having to read the whole codebase first. (*design + code*)
2. **Catch mistakes early and strengthen tests** (deprecated APIs, anti-patterns, missing or weak tests) before code review. (*test / quality*)
3. **Fit the path to production**, work with the team's CI and guardrails (don't pull in context or code from outside the approved boundaries you define) so a change moves safely toward deployment. (*CI / deploy*)
4. **Stay maintainable** as the library evolves, the team should be able to own and extend what you build without you.
5. **Work for more than the individual coder**: shape it so the people around engineering, PM, QA, and DevOps, can use or benefit from it too. One solution, multiple audiences. (*multi-role fit*)

How to build it

How you meet these requirements is up to you, there's no single right answer.

Credible approaches (pick one or combine): custom rules or conventions the assistant follows, a guided command or workflow for a scoped first task, exposing the library's docs as searchable context for the agent, an agent that scaffolds first-cut code and tests, or a tailored working mode. **Choosing the right tooling and approach for the job is part of the exercise.**

Use judgment, don't just check boxes. The requirements are a starting point. Be ready to say which problem is most worth solving for the customer's business, what you'd build first, what you'd skip, and what impact you'd expect. We're as interested in *what you chose to solve and why* as in how you solved it.

During the session (45 min)

We care as much about *how you think* as what you built. Be ready to:

- **Walk us through your thinking and problem-solving process**: how you approached it, what you tried, what you ruled out and why, and where you got stuck, not just the finished result
- Explain the problem you chose, why it matters to the business, and what you'd prioritize or skip

- **Run your working artifact live** (not screenshots)
- Walk through your key design choices, the tooling you picked and why, and where the artifact breaks

We'll keep it tight; you don't need to prepare a deep architecture deck.

What we're looking for

- Does your solution actually meet the requirements, and can you justify the approach and tooling you chose?
- **Judgment about what's worth solving** (business impact) and how you prioritized, not just what's solvable
- **Thinking process:** how you reasoned, the options you weighed, where you got stuck and how you worked through it, the journey weighs as much as the artifact
- Sound product and technical judgment in scoping what to build
- **SDLC breadth:** understanding of the full lifecycle (design, code, test, CI/CD, deploy) and how your solution helps the engineering team at each step, not just generating code
- **Multi-audience design:** a solution multiple roles can use together (PM, QA, DevOps), not only an individual engineer
- A runnable artifact, not slideware
- Reasoning from how your tools actually work, not documentation recall
- Honesty about limitations, "I don't know, here's how I'd find out" is a strong answer

Heads-up: you'll reuse this

If you advance, the **same artifact** is the foundation for the final challenge, you'll extend it into a customer-account scenario. We will not ask you to start over. However, you can extend, change or improve what you built if needed.

Resources

- Public documentation for whichever open-source library you choose

Questions? Reach out to your recruiter any time.